

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Source Code

<https://github.com/ashrithaxx/Deep-Learning-Single-Layer-Perceptron>

8. Plots

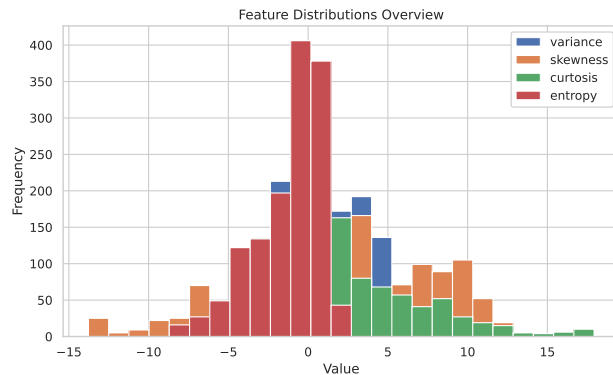


Figure 1: Feature Distribution

Interpretation: The diagram shows the distribution of the features: variance, skewness, curtosis, entropy. The features are not perfectly symmetric but there are no extreme outliers present.

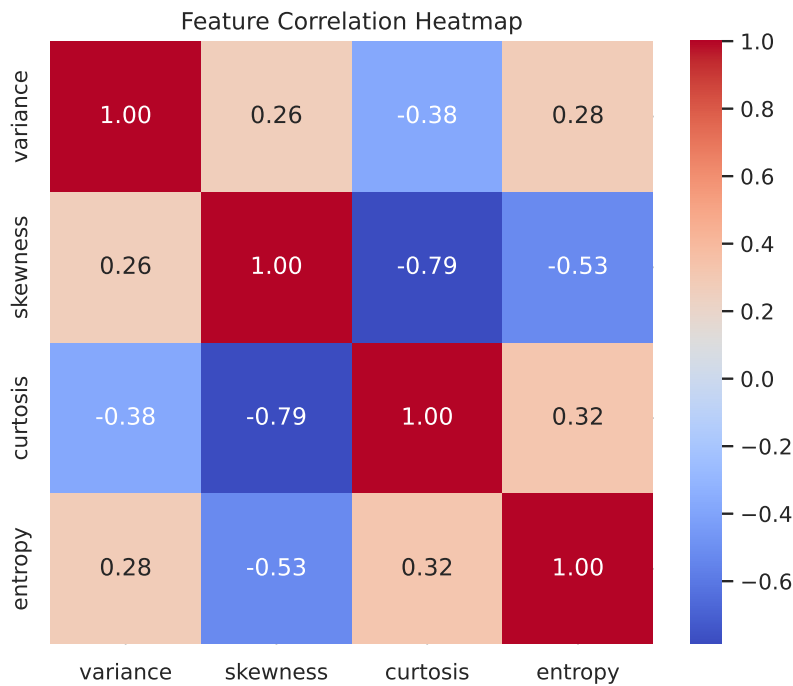


Figure 2: Feature Correlation Heatmap

Interpretation: The diagram shows the relationship strength between the features. Some features show a strong positive or negative correlation, indicating that they are related, whereas others have weak correlation indicating that they are relatively independent.

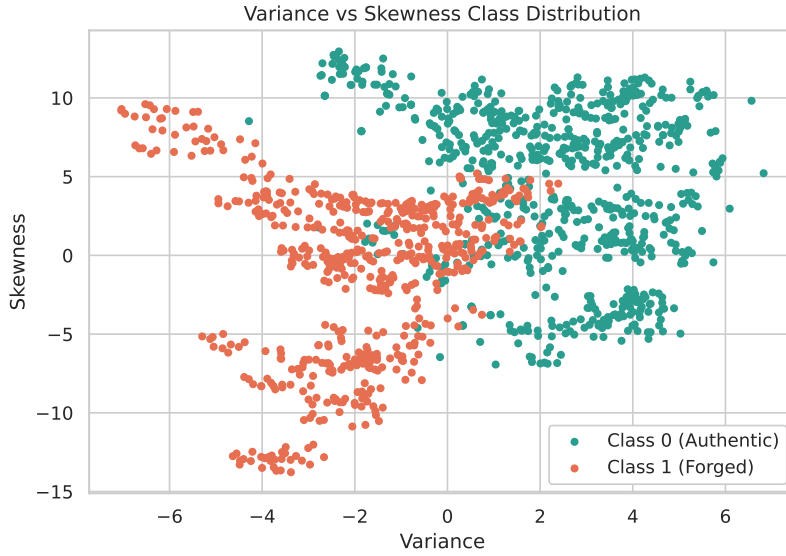


Figure 3: Variance vs Skewness Scatterplot

Interpretation: The diagram shows that the forged and authentic banknotes occupy different regions, with a small overlap, which indicates that the dataset is mostly linearly separable.

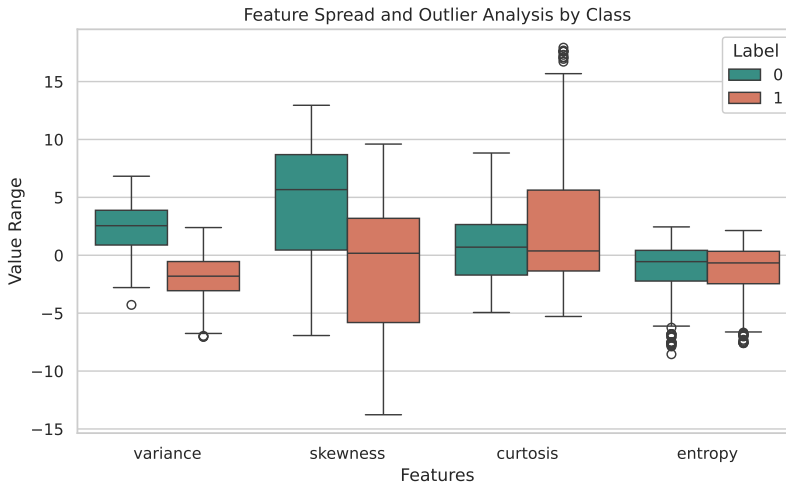


Figure 4: Feature Boxplots

Interpretation: The diagram shows the median, spread and highlights the possible outliers present. The outliers however do not dominate the dataset.

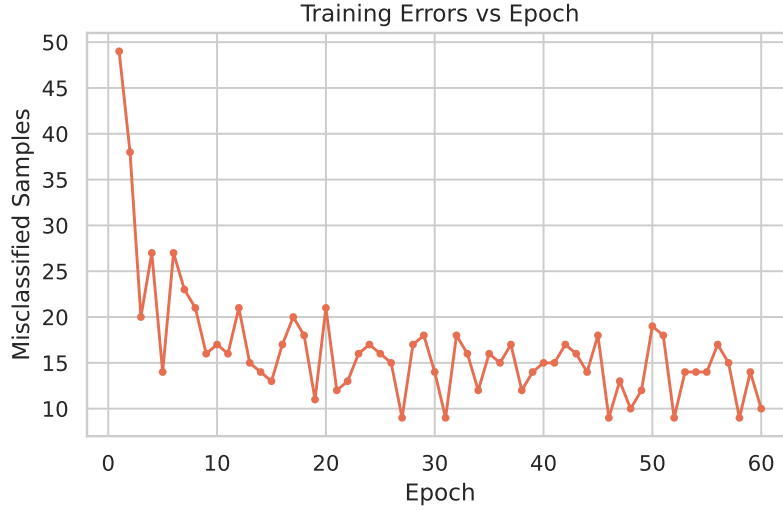


Figure 5: Training Errors vs Epoch

Interpretation: The training error decreases rapidly as the number of epochs increases. After several epochs, the error stabilizes at a low value, however there are minor fluctuations.

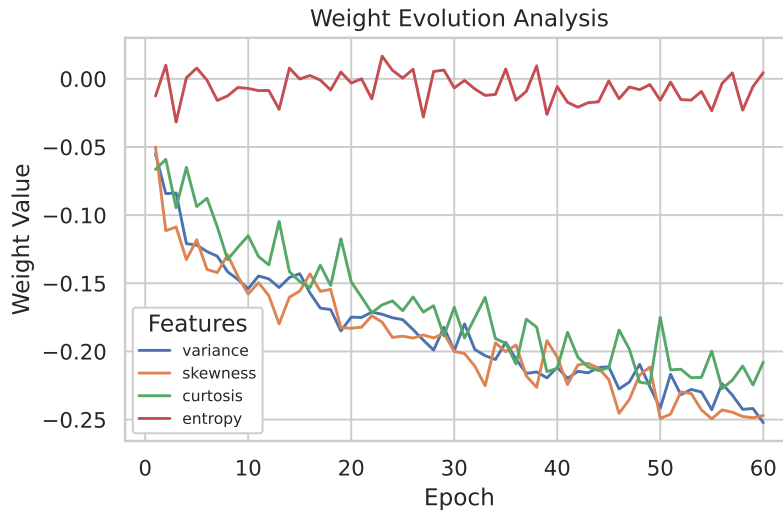


Figure 6: Weight Evolution

Interpretation: Initially, there are significant changes in the weights but as the no. of epochs increases, it stabilizes. The perceptron is learning the relative importance of the features, and is able to converge and become stable.

Interpretation: The bias is updated and has small fluctuations but is overall stable. The perceptron learns the appropriate threshold.

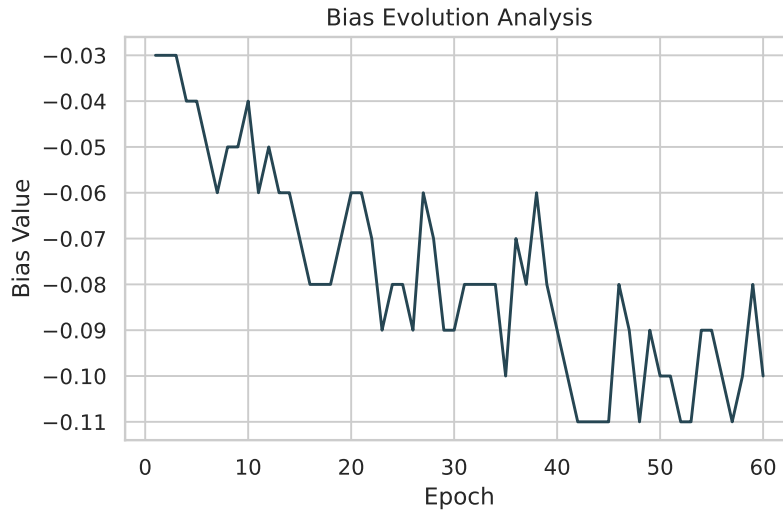


Figure 7: Bias Evolution

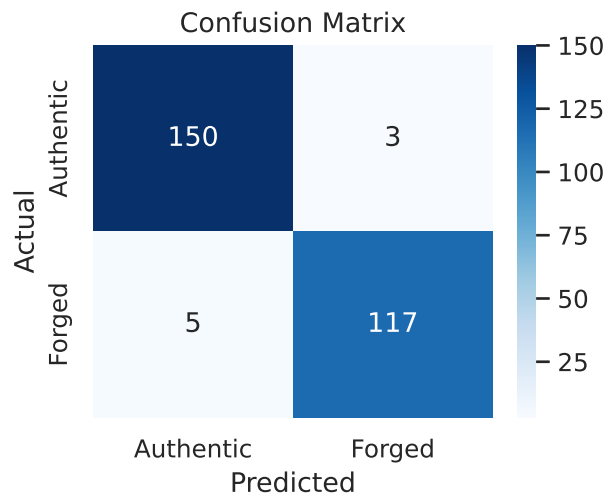


Figure 8: Confusion Matrix

Interpretation: The matrix shows that majority of the notes are classified correctly(150 authentic, 117 forged), which therefore gives us an overall higher accuracy.

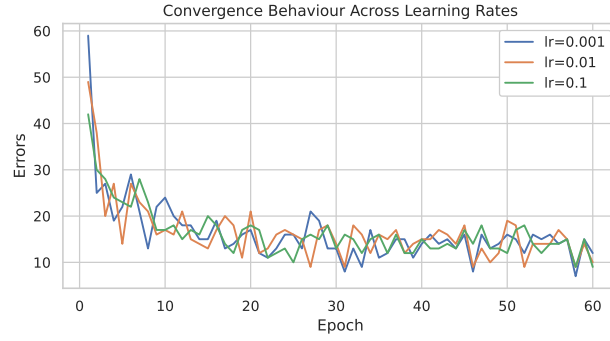


Figure 9: Learning Rate Comparison

Interpretation: Different learning rates produce different behaviors. The large learning rate fluctuates before converging.

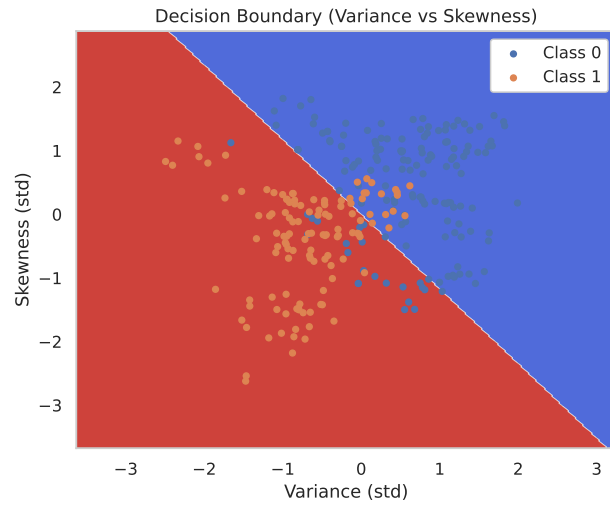


Figure 10: Decision Boundary

Interpretation: The diagram shows that authentic and forged banknotes are separated (based on variance and skewness). Even though there is a slight overlap, overall the classifier separates the two classes effectively.

9. Performance Tables

Training Summary

Dataset Size	1372
Train/Test Split	80%/20% (1097 Train / 275 Test)
Learning Rate	0.01
Epochs	60
Final Weights	[-0.252 -0.247 -0.208 0.005]
Final Bias	-0.1000
Accuracy	0.97
Precision	0.97
Recall	0.96
F1-score	0.97

Epoch-wise Learning

Epoch	Errors	Bias	Weight 1	Weight 2	Weight 3	Weight 4
1	49	-0.0300	-0.055	-0.050	-0.067	-0.013
10	17	-0.0400	-0.154	-0.158	-0.115	-0.007
20	21	-0.0600	-0.175	-0.183	-0.149	-0.003
30	14	-0.0900	-0.199	-0.200	-0.168	-0.007
40	15	-0.0900	-0.212	-0.204	-0.212	-0.006
50	19	-0.1000	-0.242	-0.249	-0.175	-0.016
60	10	-0.1000	-0.252	-0.247	-0.208	0.005

10. Discussion

1. The step function gives 0 or 1 as the output whereas the sigmoid function gives any output from 0 to 1, ie, smooth output. As step function is not differentiable at the threshold, it cannot support learning in multilayer networks, and therefore is not used in modern deep learning models.
2. Both the manual as well as the Scikit-learn are based on the perceptron learning rule. However, Scikit-learn is more optimized and handles edge cases better.
3. When $LR = 0.001$, learning happens very slowly and larger no. of epochs is required to converge. When $LR = 0.01$, it reaches a stable result within a good no. of epochs. When $LR = 0.1$, convergence occurs faster, but there will be more fluctuations.
4. XOR is not linearly seperable, and since the single-layer perceptron can only draw a linear boundary, it cannot handle XOR.
5. Without normalisation, larger range features like variance will dominate, making convergence slower. However, after normalising, it converges faster and becomes more stable, thus increasing accuracy.

11. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

12. Conclusion

The Perceptron was trained and implemented manually on the Banknote Authentication dataset, with accuracy of 97% after 60 epochs showing that the data is linearly separable.

13. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.